

# CPSC 2010: Introduction to Computer Science

**Running Time of Sorting Algorithms** April 15, 2026

## Morning quiz

**Instructions:** Go to today's quiz on Canvas, and enter access code **bubble**



Investigate the running time of bubble sort.

6 5 3 1 8 7 2 4

**Feel free to talk with students sitting near you, then submit your answers.**

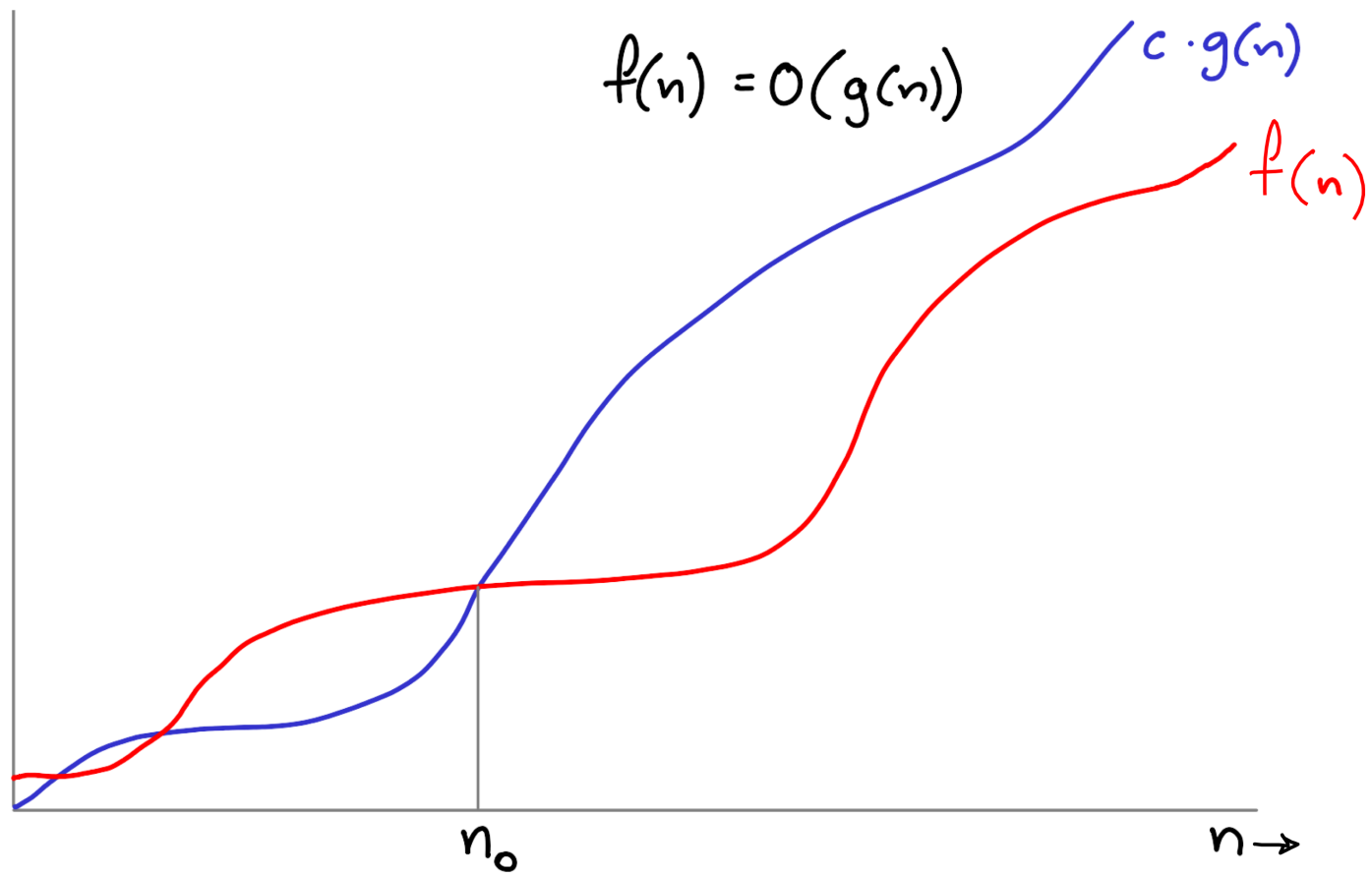
```
procedure bubbleSort(A : list of sortable items)
  n := length(A)
  repeat
    swapped := false
    for i := 1 to n-1 inclusive do
      ; If A[i-1] and A[i] are out of order, swap them
      if A[i-1] > A[i] then
        swap(A[i-1], A[i])
        swapped := true
      end if
    end for
  until not(swapped)
end procedure
```

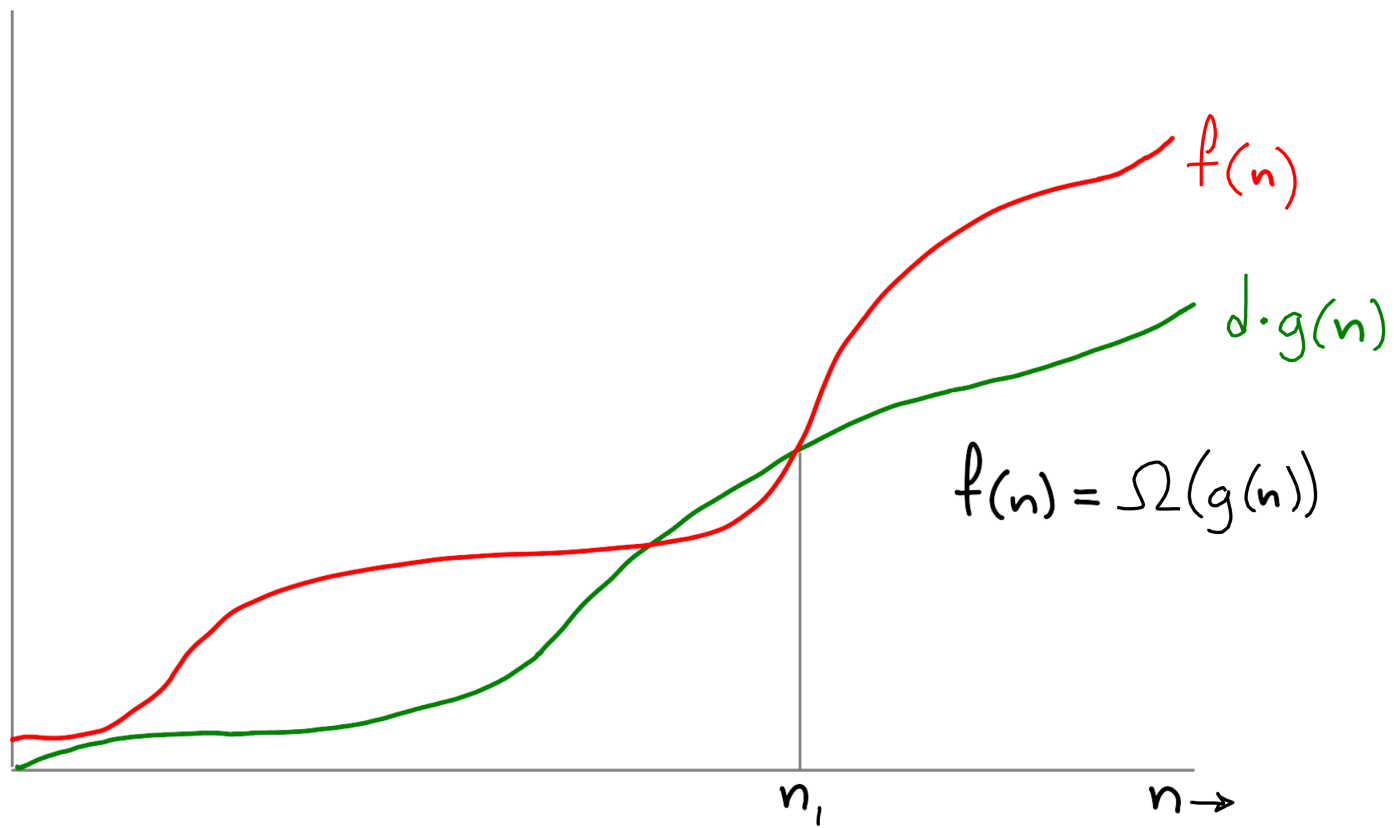
**[5, 4, 3, 2, 1]**

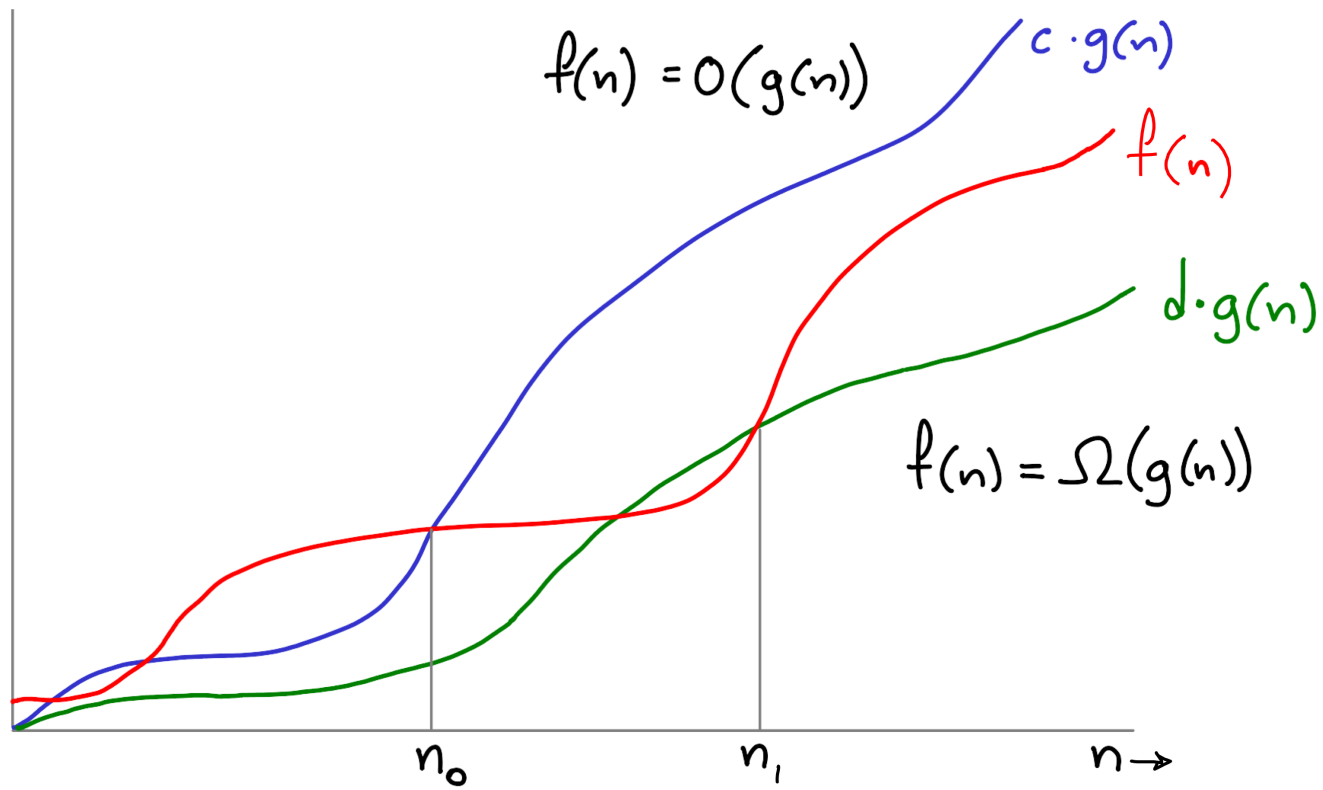
# Problems

- \*Computing the exact number is tedious (unfeasible for big code)
- \*Exact number of operations depends on many parameters
  - \* Compiler (under the hood improvements)
  - \* Computer that it is run (allocate variables in register? RAM?)
  - \* Operating System
- \*Sometimes it is hard to even compute
  - \* How much time does a search take?

Impractical!







Polynomials:  $a + bn + cn^2 + dn^3 \dots + \underline{zn^k} = O(\underline{n^k})$

$a, b, c, d, \dots, z$  : constants

Also assuming one of each term (compare to:  $a_1n + a_2n + \dots + a_n n$ )

Logarithms:  $50 \cdot \log n^3 + \log^{20} n + \underline{n^{0.1}} = O(\underline{n^{0.1}})$

"weaker" than polynomial

Exponential:  $100 \cdot n^{50} + \underline{3^n} + 40 \cdot 2^n = O(\underline{3^n})$

"stronger" than polynomial



```
void silly(int n) {  
    for (int i = 0; i < n; ++i) {  
        for (int j = 0; j < i; ++j) {  
            System.out.println("j = " + j);  
        }  
        for (int k = 0; k < n * 3; ++k) {  
            System.out.println("k = " + k);  
        }  
    }  
}
```

Runtime:

$O(n^2)$

```
void silly(int n, int x, int y) {  
    for (int i = 0; i < n; ++i) {  
        if (x < y)  
            for (int k = 0; k < n * n; ++k) {  
                System.out.println("k = " + k);  
            }  
        else  
            System.out.println("i = " + i);  
    }  
}
```

$O(n^3)$

```
void silly(int n) {  
    if (n <= 0) return;  
    System.out.println("n = " + n);  
    silly(n-1);  
}
```

$O(n)$

```
void silly(int n) {  
    if (n <= 0) return;  
    System.out.println("n = " + n);  
    silly(n/2);  
}
```

$O(\log n)$